p0798R2
Monadic operations for std::optional

Published Proposal, 2018-10-08

This version:
wg21.tartanllama.xyz/monadic-optional

Author:
Simon Brand

Audience:
LEWG, SG14

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

std::optional will be a very important vocabulary type in C++17 and up. Some uses of it can be very verbose and would benefit from operations which allow functional composition. I propose adding map, and_then, and or_else member functions to std::optional to support this monadic style of programming.

Table of Contents

§ 1. Changes from r1

- Add list of programming languages to proposed solution section
- Change `map` example code to not take address of stdlib function
- Update wording to use `std::monostate` on `void` returning callables

§ 2. Changes from r0

- More notes on P0650
- Discussion about mapping of functions returning `void`

§ 3. Motivation

`std::optional` aims to be a "vocabulary type", i.e. the canonical type to represent some programming concept. As such, `std::optional` will become widely used to represent an object which may or may not contain a value. Unfortunately, chaining together many computations which may or may not produce a value can be verbose, as empty-checking code will be mixed in with the actual programming logic. As an example, the following code automatically extracts cats from images and makes them more cute:

```
1 image get_cute_cat (const image& img) {
2     return add_rainbow(
3               make_smaller(
4                 make_eyes_sparkle(
5                   add_bow_tie(
6                     crop_to_cat(img))));
7 }
```

But there’s a problem. What if there’s not a cat in the picture? What if there’s no good place to add a bow tie? What if it has its back turned and we can’t make its eyes sparkle? Some of these operations could fail.

One option would be to throw exceptions on failure. However, there are many code bases which do not use exceptions for a variety of reasons. There’s also the possibility that we’re going to get *lots* of pictures without cats in them, in which case we’d be using exceptions for control flow. This is commonly seen as bad practice, and has an item warning against it in the C++ Core Guidelines.

Another option would be to make those operations which could fail return a `std::optional`:

```
std::optional<image> get_cute_cat (const image& img) {
    auto cropped = crop_to_cat(img);
    if (!cropped) {
      return std::nullopt;
    }

    auto with_tie = add_bow_tie(*cropped);
    if (!with_tie) {
      return std::nullopt;
    }

    auto with_sparkles = make_eyes_sparkle(*with_tie);
    if (!with_sparkles) {
      return std::nullopt;
    }

    return add_rainbow(make_smaller(*with_sparkles));
}
```

Our code now has a lot of boilerplate to deal with the case where a step fails. Not only does this increase the noise and cognitive load of the function, but if we forget to put in a check, then suddenly we're down the hole of undefined behaviour if we `*empty_optional`.

Another possibility would be to call `.value()` on the optionals and let the exception be thrown and caught like so:

```
std::optional<image> get_cute_cat (const image& img) {
    try {
        auto cropped = crop_to_cat(img);
        auto with_tie = add_bow_tie(cropped.value());
        auto with_sparkles = make_eyes_sparkle(with_tie.value());
        return add_rainbow(make_smaller(with_sparkles.value()));
    catch (std::bad_optional_access& e) {
        return std::nullopt;
    }
}
```

Again, this is using exceptions for control flow. There must be a better way.

§ 4. Proposed solution

This paper proposes adding additional member functions to `std::optional` in order to push the handling of empty states off to the side. The proposed additions are `map`, `and_then` and `or_else`. Using these new functions, the code above becomes this:

```
std::optional<image> get_cute_cat (const image& img) {
    return crop_to_cat(img)
           .and_then(add_bow_tie)
           .and_then(make_eyes_sparkle)
           .map(make_smaller)
           .map(add_rainbow);
}
```

We've successfully got rid of all the manual checking. We've even improved on the clarity of the non-optional example, which needed to either be read inside-out or split into multiple declarations.

This is common in other programming languages. Here is a list of programming languages which have a `optional`-like type with a monadic interface or some similar syntactic sugar:

- Java: `Optional`
- Swift: `Optional`
- Haskell: `Maybe`
- Rust: `Option`
- OCaml: `option`
- Scala: `Option`
- Agda: `Maybe`
- Idris: `Maybe`
- Kotlin: `T?`
- StandardML: `option`
- C#: `Nullable`

Here is a list of programming languages which have a `optional`-like type *without* a monadic interface or syntactic sugar:

- C++
- I couldn't find any others

All that we need now is an understanding of what `map` and `and_then` do and how to use them.

§ 4.1. `map`

`map` applies a function to the value stored in the optional and returns the result wrapped in an optional. If there is no stored value, then it returns an empty optional.

For example, if you have a `std::optional<std::string>` and you want to get a `std::optional<std::size_t>` giving the size of the string if one is available, you could write this:

```
1 auto s = opt_string.map([](auto&& s) { return s.size(); });
```

which is somewhat equivalent to:

```
1 if (opt_string) {
2     std::size_t s = opt_string->size();
3 }
```

`map` has one overload (expositional):

```
1 template <class T>
2 class optional {
3     template <class Return>
4     std::optional<Return> map (function<Return(T)> func);
5 };
```

It takes any callable object (like a function). If the `optional` does not have a value stored, then an empty optional is returned. Otherwise, the given function is called with the stored value as an argument, and the return value is returned inside an `optional`.

If you come from a functional programming or category theory background, you may recognise this as a functor map.

§ 4.2. `and_then`

`and_then` is like `map`, but it is used on functions which may not return a value.

For example, say you have `std::optional<std::string>` and a function like `std::stoi` which returns a `std::optional<int>` instead of throwing on failure. Rather than manually checking the optional string before calling, you could do this:

```
1 std::optional<int> i = opt_string.and_then(stoi);
```

Which is roughly equivalent to:

```
1 if (opt_string) {
2    std::optional<int> i = stoi(*opt_string);
3 }
```

`and_then` has one overload which looks like this (again, expositional):

```
1 template <class T>
2 class optional {
3     template <class Return>
4     std::optional<Return> and_then (function<std::optional<Return>(T)> fu
5 };
```

It takes any callable object which returns a `std::optional`. If the optional does not have a value stored, then an empty optional is returned. Otherwise, the given function is called with the stored value as an argument, and the return value is returned.

Again, those from an FP background will recognise this as a monadic bind.

§ 4.3. `or_else`

`or_else` returns the optional if it has a value, otherwise it calls a given function. This allows you do things like logging or throwing exceptions in monadic contexts:

```
1 get_opt().or_else([]{std::cout << "get_opt failed";});
2 get_opt().or_else([]{throw std::runtime_error("get_opt_failed")});
```

Users can easily abstract these patterns if they are common:

```
void opt_log(std::string_view msg) {
     return [=] { std::cout << msg; };
}

void opt_throw(std::string_view msg) {
     return [=] { throw std::runtime_error(msg); };
}

get_opt().or_else(opt_log("get_opt failed"));
get_opt().or_else(opt_throw("get_opt failed"));
```

It has one overload (expositional):

```
template <class T>
class optional {
    template <class Return>
    std::optional<T> or_else (function<Return()> func);
};
```

`func` will be called if `*this` is empty. `Return` will either be convertible to `std::optional<T>`, or `void`. In the former case, the return of `func` will be returned from `or_else`; in the second case `std::nullopt` will be returned.

§ 4.4. Chaining

With these two functions, doing a lot of chaining of functions which could fail becomes very clean:

```
std::optional<int> foo() {
    return
      a().and_then(b)
         .and_then(c)
         .and_then(d)
         .and_then(e);
}
```

Taking the example of `stoi` from earlier, we could carry out mathematical operations on the result by just adding `map` calls:

```
std::optional<int> i = opt_string
                       .and_then(stoi)
                       .map([](auto i) { return i * 2; });
```

We can also intersperse our chain with error checking code:

```
std::optional<int> i = opt_string
                       .and_then(stoi)
                       .or_else(opt_throw("stoi failed"))
                       .map([](auto i) { return i * 2; });
```

§ 5. How other languages handle this

`std::optional` is known as `Maybe` in Haskell and it provides much the same functionality. `map` is in `Functor` and named `fmap`, and `and_then` is in `Monad` and named `>>=` (bind).

Rust has an `Option` class, and uses the same names as are proposed in this paper. It also provides many additional member functions like `or`, `and`, `map_or_else`.

§ 6. Considerations

§ 6.1. Mapping functions returning `void`

There are three main options for handling `void` returning functions for `map`. The simplest is to disallow it (this is what `boost::optional` does). One is to return `std::optional<std::monostate>` (or some other unit type) instead of `std::optional<void>` (this is what `tl::optional` does). Another option is to add a `std::optional<void>` specialization. This proposal uses the second approach. This functionality can be desirable since it allows chaining functions which are used purely for side effects.

```
get_optional()          // returns std::optional<T>
  .map(print_value)     // returns std::optional<std::monostate>
  .map(notify_success); // Is only called when get_optional returned a va
```

§ 6.2. More functions

Rust's Option class provides a lot more than `map`, `and_then` and `or_else`. If the idea to add these is received favourably, then we can think about what other additions we may want to make.

§ 6.3. `map` only

It would be possible to merge all of these into a single function which handles all use cases. However, I think this would make the function more difficult to teach and understand.

§ 6.4. Operator overloading

We could provide operator overloads with the same semantics as the functions. For example, `|` could mean `map`, `>=` `and_then`, and `&` `or_else`. Rewriting the original example gives this:

```
1  // original
2  crop_to_cat(img)
3    .and_then(add_bow_tie)
4    .and_then(make_eyes_sparkle)
5    .map(make_smaller)
6    .map(add_rainbow);
7
8  // rewritten
9  crop_to_cat(img)
10   >= add_bow_tie
11   >= make_eyes_sparkle
12    | make_smaller
13    | add_rainbow;
```

Another option would be `>>` for `and_then`.

§ 6.5. Applicative Functors

`map` could be overloaded to accept callables wrapped in `std::optionals`. This fits the *applicative functor* concept. It would look like this:

```
template <class Return>
std::optional<Return> map (std::optional<function<Return(T)>> func);
```

This would give functional programmers the set of operations which they may expect from a monadic-style interface. However, I couldn't think of many use-cases of this in C++. If some are found then we could add the extra overload.

6.6. Alternative names

`map` may confuse users who are more familiar with its use as a data structure, or consider the common array map from other languages to be different from this application. Some other possible names are `then`, `when_value`, `fmap`, `transform`.

Alternative names for `and_then` are `bind`, `compose`, `chain`.

`or_else` could be named `catch_error`, in line with P0650r0

6.7. Overloaded `and_then`

Instead of an additional `or_else` function, `and_then` could be overloaded to take an additional error function:

```
o.and_then(a, []{throw std::runtime_error("oh no");});
```

The above will throw if `a` returns an empty optional.

7. Pitfalls

Users may want to write code like this:

```
std::optional<int> foo(int i) {
    return
      a().and_then(b)
         .and_then(get_func(i));
}
```

The problem with this is `get_func` will be called regardless of whether `b` returns an empty `std::optional` or not. If it has side effects, then this may not be what the user wants.

One possible solution to this would be to add an additional function, `bind_with` which will take a callable which provides what you want to bind to:

```
1 std::optional<int> foo(int i) {
2     return
3       a().and_then(b)
4          .bind_with([i](){return get_func(i)});
5 }
```

§ 7.1. Other solutions

There is a proposal for adding a general monadic interface to C++. Unfortunately doing the kind of composition described above would be very verbose with the current proposal without some kind of Haskell-style `do` notation. The code for my first solution above would look like this:

```
1  std::optional<int> get_cute_cat(const image& img) {
2      return
3        functor::map(
4          functor::map(
5            monad::bind(
6              monad::bind(crop_to_cat(img),
7                add_bow_tie),
8              make_eyes_sparkle),
9            make_smaller),
10       add_rainbow);
11 }
```

My proposal is not necessarily an alternative to this proposal; compatibility between the two could be ensured and the generic proposal could use my proposal as part of its implementation. This would allow users to use both the generic syntax for flexibility and extensibility, and the member-function syntax for brevity and clarity.

If `do` notation or unified call syntax is accepted, then my proposal may not be necessary, as use of the generic monadic functionality would allow the same or similarly concise syntax.

Another option would be to use a ranges-style interface for the general monadic interface:

```
std::optional<int> get_cute_cat(const image& img) {
    return crop_to_cat(img)
         | monad::bind(add_bow_tie)
         | monad::bind(make_eyes_sparkle)
         | functor::map(make_smaller)
         | functor::map(add_rainbow);
}
```

§ 7.2. Interaction with other proposals

There is a proposal for std::expected which would benefit from many of these same ideas. If the idea to add monadic interfaces to standard library classes on a case-by-case basis is chosen rather than a unified non-member function interface, then compatibility between this proposal and the `std::expected` one should be maintained.

Mapping functions which return `void` can be supported, but is a pain to implement since `void` is not a regular type. If the Regular Void proposal was accepted, implementation would be simpler and the results of the operation would conform to users' expectations better.

Any proposals which make lambdas or overload sets easier to write and pass around will greatly improve this proposal. In particular, proposals for lift operators and abbreviated lambdas would ensure that the clean style is preserved in the face of many anonymous functions and overloads.

If an operator overloading approach is taken and the workflow operator (`operator|>`) is accepted, then that could be used instead of `operator|`.

§ 8. Implementation experience

This proposal has been implemented here.

✧ ✧ ✧

§ 9. Proposed Wording

§ 9.1. New synopsis entry: `[optional.monadic]`, monadic operations

```
template <class F> constexpr *see below* and_then(F&& f) &;
template <class F> constexpr *see below* and_then(F&& f) &&;
template <class F> constexpr *see below* and_then(F&& f) const&;
template <class F> constexpr *see below* and_then(F&& f) const&&;
template <class F> constexpr *see below* map(F&& f) &;
template <class F> constexpr *see below* map(F&& f) &&;
template <class F> constexpr *see below* map(F&& f) const&;
template <class F> constexpr *see below* map(F&& f) const&&;
template <class F> constexpr optional<T> or_else(F &&f) &;
template <class F> constexpr optional<T> or_else(F &&f) &&;
template <class F> constexpr optional<T> or_else(F &&f) const&;
template <class F> constexpr optional<T> or_else(F &&f) const&&;
```

§ 9.2. New section: Monadic operations `[optional.monadic]`

```
template <class F> constexpr *see below* and_then(F&& f) &;
template <class F> constexpr *see below* and_then(F&& f) const&;
```

Requires: `std::invoke(std::forward<F>(f), value())` returns a `std::optional<U>` for some `U`.

Returns: Let `U` be the result of `std::invoke(std::forward<F>(f), value())`. Returns a `std::optional<U>`. If `*this` contains a value then `std::invoke(std::forward<F>(f), value())` is returned, otherwise the returned `optional` does not contain a value.

```
template <class F> constexpr *see below* and_then(F&& f) &&;
template <class F> constexpr *see below* and_then(F&& f) const&&;
```

Requires: `std::invoke(std::forward<F>(f), std::move(value()))` returns a `std::optional<U>` for some `U`.

Returns: Let `U` be the result of `std::invoke(std::forward<F>(f), std::move(value()))`. Returns a `std::optional<U>`. If `*this` contains a value then `std::invoke(std::forward<F>`

`(f), std::move(value())` is returned, otherwise the returned `optional` does not contain a value.

```
1 template <class F> constexpr *see below* map(F&& f) &;
2 template <class F> constexpr *see below* map(F&& f) const&;
```

Returns: Let `U` be the result of `std::invoke(std::forward<F>(f), value())`.

- If `U` is `void`, returns a `std::optional<std::monostate>`. If `*this` contains a value then `f` is invoked as if by `std::invoke(std::forward<F>(f), value())` and the returned `optional` will contain a default-constructed `std::monostate` value; otherwise the returned `optional` will not contain a value.
- If `U` is not `void`, returns a `std::optional<U>`. If `*this` contains a value then an `optional<U>` is constructed from `std::invoke(std::forward<F>(f), value())` and returned, otherwise the returned `optional` does not contain a value.

```
1 template <class F> constexpr *see below* map(F&& f) &&;
2 template <class F> constexpr *see below* map(F&& f) const&&;
```

Returns: Let `U` be the result of `std::invoke(std::forward<F>(f), std::move(value()))`.

- If `U` is `void`, returns a `std::optional<std::monostate>`. If `*this` contains a value then `f` is invoked as if by `std::invoke(std::forward<F>(f), std::move(value()))` and the returned `optional` will contain a default-constructed `std::monostate` value; otherwise the returned `optional` will not contain a value.
- If `U` is not `void`, returns a `std::optional<U>`. If `*this` contains a value then an `optional<U>` is constructed from `std::invoke(std::forward<F>(f), std::move(value())` and returned, otherwise the returned `optional` does not contain a value.

```
1 template <class F> constexpr optional<T> or_else(F &&f) &;
2 template <class F> constexpr optional<T> or_else(F &&f) const&;
```

Requires: `std::invoke_result_t<F>` must be `void` or convertible to `optional<T>`.

Effects: If `*this` has a value, returns `*this`. Otherwise, if `f` returns void, calls `std::forward<F>(f)` and returns `std::nullopt`. Otherwise, returns `std::forward<F>(f)()`;

```
template <class F> constexpr optional<T> or_else(F &&f) &&;
template <class F> constexpr optional<T> or_else(F &&f) const&&;
```

Requires: `std::invoke_result_t<F>` must be `void` or convertible to `optional<T>`.

Effects: If `*this` has a value, returns `std::move(*this)`. Otherwise, if `f` returns void, calls `std::forward<F>(f)` and returns `std::nullopt`. Otherwise, returns `std::forward<F>(f)()`;

✧ ✧ ✧

§ 9.3. Acknowledgements

Thank you to Michael Wong for representing this paper to the committee. Thanks to Kenneth Benzie, Vittorio Romeo, Jonathan Müller, Adi Shavit, Nicol Bolas, Vicente Escribá and Barry Revzin for review and suggestions.